



Programación declarativa y funcional

Streams y Lambdas

Paradigmas de programación

- **Imperativos:** Procedimental, Estructurado.
- **Declarativos:** Funcional, Lógico, Relacional.
- **Orientados a Objetos:** Clásico, Basado en prototipos.
- **Multiproceso:** Concurrente, Paralelo, Distribuido.
- **Basados en Flujos de Datos y/o Eventos:** Funcional, Visual, Reactivo
- **Basados en Reglas :** Programación lógica.
- **Metaprogramación:** Reflexivo.
- **Otros paradigmas:** Programación orientada a aspectos, Computación evolutiva, Computación neuronal, Programación literaria, Programación simbólica, etc.

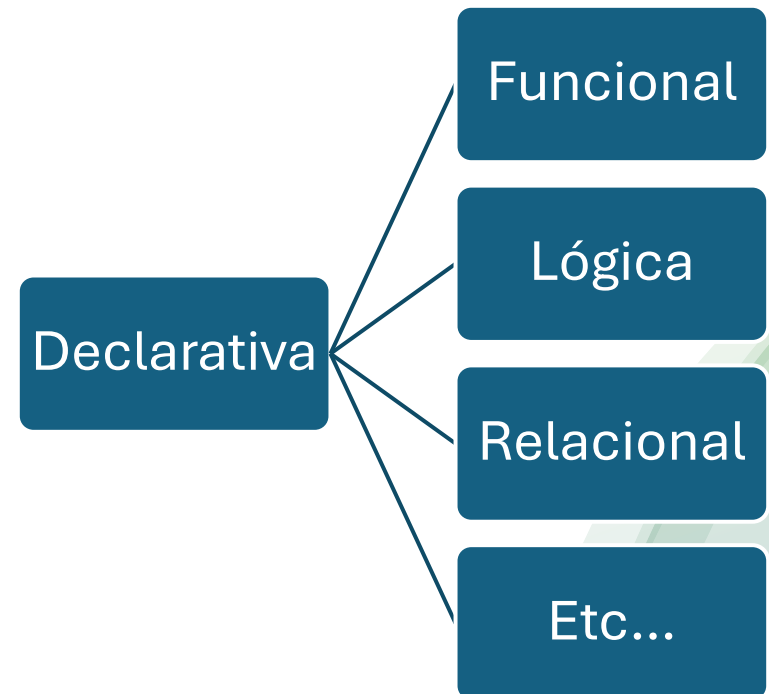
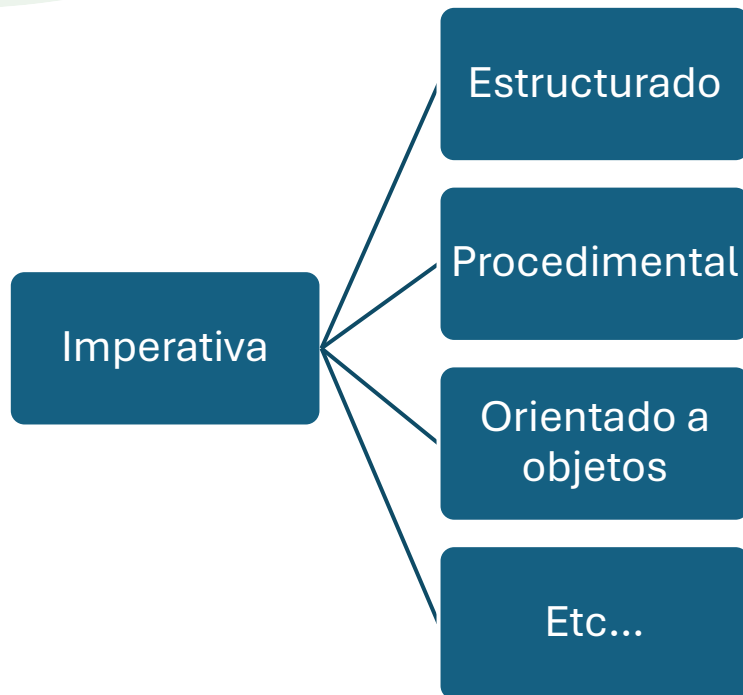
Enfoque Imperativo vs Declarativo

Programación Imperativa: Especifica el “Cómo” se deben resolver los problemas. Utiliza principalmente instrucciones de control de flujo.

Programación Declarativa: Especifica el “Qué” se desea calcular/obtener a través de la descripción de la lógica de computación que permite resolver los problemas.

Programación Funcional

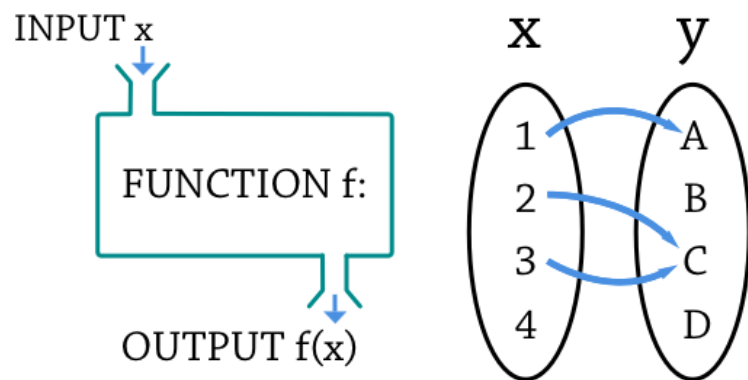
Basado fundamentalmente en: la composición de funciones, la recursividad y como era esperable, las funciones, pero tanto en su papel de módulos “funcionales” como en su papel de argumentos de otras funciones.



Programación Funcional

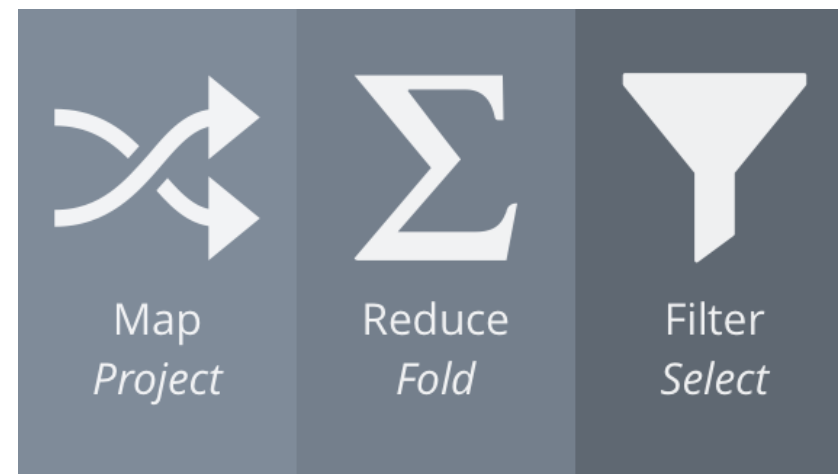
Función

Relación entre conjuntos.



Función de orden superior

Admiten otras funciones como argumentos.



Transforma un valor en otro

Aplica función de agregación

Filtra según un criterio

Programación Funcional

Enfoque

```
List<Integer> numeros = Arrays.asList(1,2,3,4,5);
List<Integer> numerosDoble = new ArrayList<>();

// Procesar
for (int numero : numeros)
    numerosDoble.add(numero * 2);

// Imprimir
for (int numero : numerosDoble)    System.out.println(numero);
```

Enfoque funcional (Streams y lambda)

```
List<Integer> numeros = Arrays.asList(1,2,3,4,5);
// Procesar
List<Integer> numerosDoble = numeros.stream()
                                    .map(numero -> numero * 2)
                                    .collect(Collectors.toList());

// Imprimir
numerosDoble.forEach(System.out::println);
```

Interfaces Funcionales

- Son aquellos que tienen un único método abstracto (sin implementar). Puede tener otros métodos por defecto.

```
@FunctionalInterface
public interface InterfaceFuncionalEducado
{
    public String saludar(String nombre); // Método abstracto
    public default String saludoUniversal() {return "Hola a todos";}
}
```

```
InterfaceFuncionalEducado ifc = (name) -> "Hola " +name; // Implementación
```

Identificador

Argumentos del
método abstracto

Cuerpo del método
abstracto

```
System.out.println(ifc.saludar("Pepe"));
System.out.println(ifc.saludoUniversal());
```

Lambda Expresion (λ)

(name) -> "Hola " +name

Implementa

Interface Funcional

Método "Default"

return "Hola a todos"

Método Abstracto {}

Streams

- Secuencias de elementos que admiten enfoque **funcional-declarativo**.
- Soportan procesamiento **secuencial ó paralelo** (multiprogramación).
- Suelen crearse a partir de **colecciones**. Admiten también Arrays.
- El uso frecuente es bajo el esquema de tubería o “***pipeline***”.
- Son **inmutables**. Los datos originales no cambian.
- Encadenamiento seguro: Cada operación en un stream retorna un nuevo stream. **No son reutilizables**.

Tipos de operaciones

- Intermedias: Devuelven un stream. Ej. filter(), sorted(), map().
- Terminales: Devuelven resultado final. Ej:forEach(), collect(), reduce()

```
List<String> names = Arrays.asList("Ana", "Luis", "Carlos", "Lucía", "Arturo",  
    "Laura", "Andrés", "Leonardo");  
  
List<String> result = names.stream()  
    .filter(name -> name.startsWith("L") || name.startsWith("A"))  
    .filter(name -> name.length() > 4)  
    .map(String::toUpperCase)  
    .sorted()  
    .collect(Collectors.toList());  
  
System.out.println("Nombres seleccionados: " + result);  
  
System.out.println("Nombres con foreach:");  
result.forEach(System.out::println);  
  
// Resultado: ["ANDRÉS", "ARTURO", "LAURA", "LEONARDO", "LUCÍA"]
```

Lazy evaluation (evaluación perezosa)

- Las operaciones se aplazan hasta que se requiere un resultado terminal pues estos “activan” el pipeline.

```
List<String> names = Arrays.asList("Ana", "Luis", "Carlos", "Lucía", "Arturo");

// Definimos el pipeline (operaciones intermedias)
names.stream().filter(name -> {System.out.println("Filtrando: " + name);
                                return name.length() > 4;
                                })
    .map(name -> {System.out.println("Mapeando: " + name);
                  return name.toUpperCase();
                }); // Aquí no sucede nada. No hay operación terminal.

// Definimos incluyendo operación terminal (activa el pipeline)
names.stream().filter(name -> {System.out.println("Filtrando: " + name);
                                return name.length() > 4;
                                })
    .map(name -> {System.out.println("Mapeando: " + name);
                  return name.toUpperCase();
                });
    .forEach(System.out::println); // Aquí se procesan los datos.
```

Streams: Operaciones intermedias.

Filtrado: `lista.stream ().filter (s -> s.startsWith("a"))`

Ordenación natural: `lista.stream ().sorted()`

Ordenación externa: `lista.stream ().sorted(comp)`

`Comparator<tipo> comp = (x,y) -> x.nombre.compareTo(y.nombre) ;`

Transformación: `lista.stream().map(c -> c.dni)`

Eliminar repetidos: `lista.stream().distinct()`

Operaciones aritméticas: `sum(), average(), reduce(), max(), min(), skip(n),`

Streams: Operaciones terminales.

forEach: Aplica una acción a cada elemento.

```
stream.forEach(System.out::println)
```

collect: Convierte los elementos del Stream en una colección u otro tipo de resultado.

```
List<String> list = stream.collect(Collectors.toList());
```

reduce: Combina los elementos en un solo valor usando un acumulador.

```
Optional<Integer> sum = stream.reduce(Integer::sum)
```

count: Cuenta el número de elementos en el Stream.

```
long count = stream.count();
```

anyMatch / allMatch / noneMatch: Verifica si al menos un, todos, o ninguno de los elementos coinciden con una condición dada.

```
boolean anyStartsWithA = stream.anyMatch(s -> s.startsWith("A"));
```

Funciones lambda λ

- Una expresión lambda (***lambda expression***) es una función anónima.
- Por su naturaleza se define e inmediatamente se usa.
- Escenario de desacoplamiento:
 - Clase contenedora de elementos a los que queremos aplicar una operación que no conoce.
 - La clase cliente (la que usa a la contenedora) no sabe iterar sobre el conjunto.
- Básicamente consiste en declarar y enviar una función como argumento a otra función.

Funciones lambda λ

```
1  class ValueSet {  
2      private Set<BigDecimal> values = new HashSet<>();  
3      public BigDecimal applyAndSum(Function<BigDecimal, BigDecimal> function) {  
4          return values.stream()  
5              .map(function::apply)  
6              .reduce(BigDecimal.ZERO, BigDecimal::add);  
7      }  
8  }  
9  public static void main(String[] args) {  
10     ValueSet values = new ValueSet();  
11     values.applyAndSum(value -> value.multiply(BigDecimal.valueOf(10)));  
12 }
```

La parte del código cliente puede pasar un bloque de código para que la clase contenedora lo aplique sobre los elementos utilizando una expresión lambda.

Interface funcional, lambda y stream

Interface func.	Método abstracto	Ejemplo
Consumer<T>	void accept(T t)	.forEach (name -> System.out.println(name))
Predicate<T>	boolean test<T x>	.filter(name -> name.length()>4)
Function<T R>	R apply <T x>	.map(name -> name.length())
BiFunction<T U R>	R apply<T x U y>	.map((a,b) -> a/b)
Supplier	T get	.map(name -> prefijor.get()+name) (+) <i>Ver ejemplo</i>

```
List<String> names = Arrays.asList("ana", "luis", "carlos", "lucía", "arturo");

Supplier<String> prefixSupplier = () -> "Sr/Sra: ";
names.stream()
    .filter(name -> name.length() > 4)
    .map(name -> name.substring(0,1).toUpperCase() + name.substring(1))
    .map(name -> prefixSupplier.get() + name)
    .sorted()
    .forEach(System.out::println);
```

Operador ::

- El Operador :: se conoce como operador de referencia a métodos. Permite referenciar métodos o constructores de una manera más concisa y legible.

```
// Lambda tradicional  
list.forEach(item -> System.out.println(item));  
  
// Usando referencia a método  
list.forEach(System.out::println);
```


Operador ::

Referencia a un método estático Clase::métodoEstático

```
public class MiClase
{
    public static void imprimir(String mensaje)
    {
        System.out.println(mensaje);
    }

    public static void main(String[] args)
    {
        List<String> mensajes = Arrays.asList("Hola", "Mundo");
        mensajes.forEach(MiClase::imprimir);
    }
}
```

Operador ::

Referencia a un método de instancia de un objeto específico
instancia::métodoDeInstancia

```
public class MiClase
{
    public void imprimir(String mensaje)
    {
        System.out.println(mensaje);
    }

    public static void main(String[] args)
    {
        MiClase ejemplo = new MiClase ();
        List<String> mensajes = Arrays.asList("Hola", "Mundo");
        mensajes.forEach(ejemplo::imprimir);
    }
}
```

Operador ::

Referencia a un método de instancia de un objeto arbitrario

Clase::métodoDeInstancia

```
public class ArbitraryMethodExample
{
    public static void main(String[] args)
    {
        List<String> mensajes = Arrays.asList("Hola", "Mundo");
        mensajes.sort(String::compareToIgnoreCase);
        mensajes.forEach(System.out::println); // Imprime mensajes ordenados
    }
}
```

Operador ::

Referencia a un constructor Clase::**new**

```
import java.util.function.Supplier;
public class MiClase
{
    public MiClase()
    {
        System.out.println("Objeto creado");
    }

    public static void main(String[] args)
    {
        // Usar un Supplier para crear un nuevo objeto
        Supplier<MiClase> constructor = MiClase::new;
        constructor.get(); // Llama al constructor y crea un objeto
    }
}
```



Programación declarativa y funcional

Streams y Lambdas